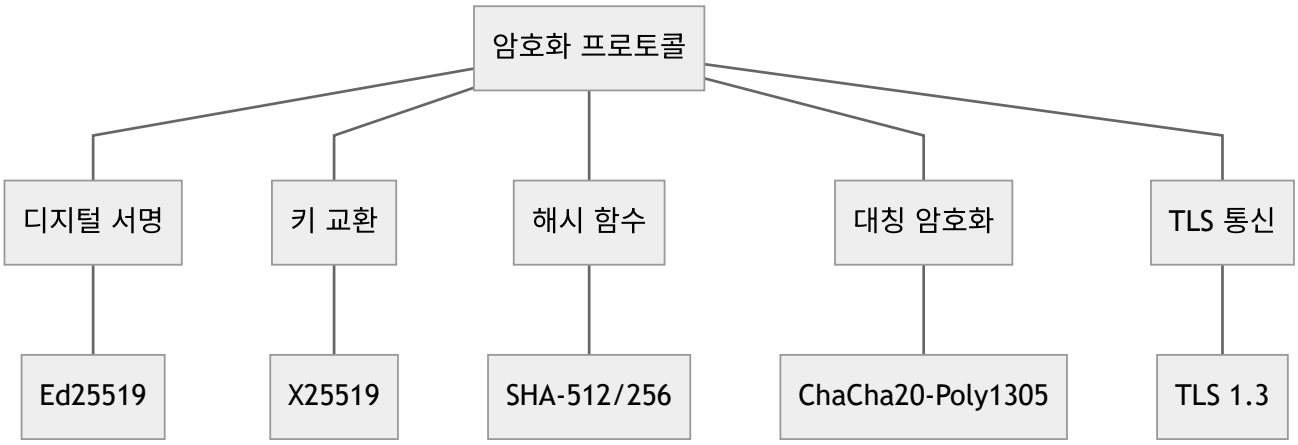
암호화 프로토콜 (Cryptographic Protocols)

소개

Oasis Core는 다양한 암호화 알고리즘과 프로토콜을 사용하여 네트워크의 보안을 보장합니다. 이 문서는 Oasis Core에서 사용되는 주요 암호화 기술들과 그 구현 방식을 상세히 설명합니다.

암호화 알고리즘 개요

1. 사용되는 암호화 알고리즘 분류



2. 암호화 상수 정의

```
const (

// 공개키 크기 (Ed25519)

PublicKeySize = ed25519.PublicKeySize  // 32 bytes

// 서명 크기 (Ed25519)
```

```
SignatureSize = ed25519.SignatureSize  // 64 bytes

// 해시 크기 (SHA-512/256)

HashSize = 32  // bytes

// X25519 키 크기

X25519KeySize = 32  // bytes

)
```

디지털 서명 시스템

1. Ed25519 서명 알고리즘

기본 구조

```
// PublicKey는 Ed25519 공개키를 나타냅니다

type PublicKey [PublicKeySize]byte


// Signature는 Ed25519 서명을 나타냅니다

type Signature [SignatureSize]byte


// Signer는 디지털 서명을 생성할 수 있는 인터페이스입니다

type Signer interface {

// Public은 서명자의 공개키를 반환합니다

Public() PublicKey

// ContextSign은 컨텍스트와 메시지에 대한 서명을 생성합니다
```

```go
ContextSign(context Context, message []byte) ([]byte, error)

// String은 서명자의 문자열 표현을 반환합니다 (민감한 정보 제외)

String() string

// Reset은 서명자를 해제하고 민감한 상태를 삭제합니다

Reset()

}
```

컨텍스트 기반 서명

```go
// Context는 도메인 분리 컨텍스트입니다

type Context string


// NewContext는 새로운 컨텍스트를 생성하고 등록합니다

func  NewContext(rawContext  string, opts ...ContextOption) Context {

// RFC 8032 호환성을 위한 길이 검증

l := len(rawContext)

if  l == 0 {

panic("빈 컨텍스트는 허용되지 않음")

}

// 체인 분리가 활성화된 경우 추가 길이 고려

if  opt.chainSeparation {

l += len(chainContextSeparator) + chainContextMaxSize

}

if  l > ed25519.ContextMaxSize {
```

```
panic("컨텍스트가 너무 김")
}
ctx := Context(rawContext)
registeredContexts.Store(ctx, &opt)
return ctx
}

// 체인별 도메인 분리 옵션
func WithChainSeparation() ContextOption {
return func(o *contextOptions) {
o.chainSeparation = true
}
}
```

서명 생성과 검증

```
// PrepareSignerMessage는 서명을 위한 컨텍스트와 메시지를 준비합니다
func PrepareSignerMessage(context Context, message []byte) ([]byte, error) {
rawContext, err := PrepareSignerContext(context)
if err != nil {
return nil, err
}
// SHA-512/256을 사용한 해시 기반 서명 (RFC 8032 대신)
```

```go
// HSM 호환성을 위해 Ed25519ph 대신 사용
h := sha512.New512_256()
_, _ = h.Write(rawContext)
_, _ = h.Write(message)
sum := h.Sum(nil)
return sum[:], nil
}

// Sign은 메시지에 대한 서명을 생성합니다
func Sign(signer Signer, context Context, message []byte) ([]byte, error) {
return signer.ContextSign(context, message)
}

// Verify는 서명을 검증합니다
func Verify(publicKey PublicKey, context Context, message []byte, sig []byte) e
if len(sig) != SignatureSize {
return ErrMalformedSignature
}
preparedMessage, err := PrepareSignerMessage(context, message)
if err != nil {
return err
}
if !ed25519.Verify(ed25519.PublicKey(publicKey[:]), preparedMessage, sig) {
```

```
return  ErrVerificationFailed
}
return  nil
}
```

2. 배치 서명 검증

성능 최적화를 위한 배치 검증

```
// BatchVerifier는 여러 서명을 효율적으로 검증합니다
type BatchVerifier struct {
entries []batchVerifyEntry
cache *cache.PartiallyPrecomputedGroupElement
}

type batchVerifyEntry struct {
publicKey PublicKey
context Context
message []byte
signature []byte
}

// Add는 배치 검증에 서명을 추가합니다
func (bv *BatchVerifier) Add(publicKey PublicKey, context Context, message []byt
```

```go
if len(sig) != SignatureSize {
return ErrMalformedSignature
}
bv.entries = append(bv.entries, batchVerifyEntry{
publicKey: publicKey,
context: context,
message: message,
signature: sig,
})
return nil
}

// Verify는 추가된 모든 서명을 배치로 검증합니다
func (bv *BatchVerifier) Verify(rng io.Reader) (bool, []int) {
if len(bv.entries) == 0 {
return true, nil
}
// Ed25519 배치 검증 알고리즘 사용
return bv.verifyBatch(rng)
}
```

키 교환 프로토콜

1. X25519 키 교환

Curve25519 기반 ECDH

```go
// X25519PublicKey는 X25519 공개키입니다
type X25519PublicKey [32]byte

// X25519PrivateKey는 X25519 개인키입니다
type X25519PrivateKey [32]byte

// GenerateX25519KeyPair는 새로운 X25519 키 쌍을 생성합니다
func GenerateX25519KeyPair(rng io.Reader) (*X25519PrivateKey, *X25519PublicKey,
var privateKey X25519PrivateKey
var publicKey X25519PublicKey
// 개인키 생성
if _, err := io.ReadFull(rng, privateKey[:]); err != nil {
return nil, nil, err
}
// 공개키 계산
curve25519.ScalarBaseMult(&publicKey, &privateKey)
return &privateKey, &publicKey, nil
}

// ComputeSharedSecret은 공유 비밀을 계산합니다
```

```go
func ComputeSharedSecret(privateKey *X25519PrivateKey, publicKey *X25519PublicK
var sharedSecret [32]byte
curve25519.ScalarMult(&sharedSecret, privateKey, publicKey)
// 약한 키 검사
if isWeakSharedSecret(&sharedSecret) {
return nil, errors.New("약한 공유 비밀 생성됨")
}
return &sharedSecret, nil
}
```

2. 키 매니저 암호화 키 관리

런타임 암호화 키 (REK) 시스템

```go
// RuntimeEncryptionKey는 런타임별 암호화 키입니다
type RuntimeEncryptionKey struct {
PublicKey X25519PublicKey `json:"public_key"`
PrivateKey X25519PrivateKey `json:"-"` // 직렬화에서 제외
Generation uint64 `json:"generation"`
Epoch beacon.EpochTime `json:"epoch"`
}

// EncryptForRuntime은 특정 런타임을 위해 데이터를 암호화합니다
func EncryptForRuntime(data []byte, runtimeKeys map[X25519PublicKey]struct{})
```

```go
// 임시 키 쌍 생성
ephemeralPriv, ephemeralPub, err := GenerateX25519KeyPair(rand.Reader)
if err != nil {
return nil, err
}
encryptedData := &EncryptedData{
EphemeralPublicKey: *ephemeralPub,
Ciphertexts: make(map[X25519PublicKey][]byte),
}
// 각 런타임 키에 대해 암호화
for runtimePubKey := range runtimeKeys {
sharedSecret, err := ComputeSharedSecret(ephemeralPriv, &runtimePubKey)
if err != nil {
return nil, err
}
// ChaCha20-Poly1305를 사용한 AEAD 암호화
ciphertext, err := encryptWithChaCha20Poly1305(data, sharedSecret[:])
if err != nil {
return nil, err
}
encryptedData.Ciphertexts[runtimePubKey] = ciphertext
}
return encryptedData, nil
```

```
}
```

해시 함수

1. SHA-512/256 해시

기본 해시 구조

```
// Hash는 임의의 바이너리 데이터에 대한 암호화 해시입니다
type Hash [Size]byte

const Size = 32 // SHA-512/256 출력 크기

// NewFromBytes는 바이트 문자열을 해시하여 새로운 해시를 생성합니다
func NewFromBytes(data ...[]byte) (h Hash) {
h.FromBytes(data...)
return
}

// FromBytes는 임의의 바이트 문자열의 해시로 설정합니다
func (h *Hash) FromBytes(data ...[]byte) {
hasher := sha512.New512_256()
for _, d := range data {
```

```
_, _ = hasher.Write(d)
}
sum := hasher.Sum([]byte{})
_ = h.UnmarshalBinary(sum[:])
}

// From은 임의의 CBOR 직렬화 가능한 인터페이스의 해시로 설정합니다
func (h *Hash) From(v any) {
h.FromBytes(cbor.Marshal(v))
}
```

스트리밍 해시 빌더

```
// Builder는 반복적으로 해시를 계산할 수 있는 해시 빌더입니다
type Builder struct {
hasher hash.Hash
}

// Write는 실행 중인 해시에 더 많은 데이터를 추가합니다
func (b *Builder) Write(p []byte) (int, error) {
return  b.hasher.Write(p)
}

// Build는 현재 해시를 반환합니다
```

```go
func (b *Builder) Build() (h Hash) {
sum := b.hasher.Sum([]byte{})
_ = h.UnmarshalBinary(sum[:])
return
}

// NewBuilder는 새로운 해시 빌더를 생성합니다
func NewBuilder() *Builder {
return &Builder{hasher: sha512.New512_256()}
}

// 사용 예제
func ComputeIncrementalHash(chunks [][]byte) Hash {
builder := NewBuilder()
for _, chunk := range chunks {
builder.Write(chunk)
}
return builder.Build()
}
```

2. Merkle Tree 해시

MKVS에서의 해시 사용

```go
// 노드 해시 계산
func (n *LeafNode) UpdateHash() {
var  keyLen, valueLen [4]byte
binary.LittleEndian.PutUint32(keyLen[:], uint32(len(n.Key)))
binary.LittleEndian.PutUint32(valueLen[:], uint32(len(n.Value)))
h := sha512.New512_256()
_, _ = h.Write([]byte("oasis-core/mkvs:leaf"))
_, _ = h.Write(keyLen[:])
_, _ = h.Write(n.Key)
_, _ = h.Write(valueLen[:])
_, _ = h.Write(n.Value)
sum := h.Sum(nil)
_ = n.Hash.UnmarshalBinary(sum[:])
}


// 내부 노드 해시 계산
func (n *InternalNode) UpdateHash() {
leafNodeHash := n.LeafNode.GetHash()
leftHash := n.Left.GetHash()
rightHash := n.Right.GetHash()
h := sha512.New512_256()
_, _ = h.Write([]byte("oasis-core/mkvs:internal"))
```

```
_, _ = h.Write(leafNodeHash[:])

_, _ = h.Write(leftHash[:])

_, _ = h.Write(rightHash[:])

sum := h.Sum(nil)

_ = n.Hash.UnmarshalBinary(sum[:])

}
```

대칭 암호화

1. ChaCha20-Poly1305 AEAD

인증된 암호화

```
// encryptWithChaCha20Poly1305는 ChaCha20-Poly1305를 사용하여 데이터를 암호화합니다

func encryptWithChaCha20Poly1305(plaintext []byte, key []byte) ([]byte, error)

if len(key) != 32 {

return nil, errors.New("키 길이는 32바이트여야 함")

}

// AEAD 암호 생성

aead, err := chacha20poly1305.New(key)

if err != nil {

return nil, err

}

// 논스 생성
```

```go
nonce := make([]byte, aead.NonceSize())
if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
return nil, err
}
// 암호화 (인증 태그 포함)
ciphertext := aead.Seal(nil, nonce, plaintext, nil)
// 논스와 암호문 결합
result := make([]byte, len(nonce)+len(ciphertext))
copy(result[:len(nonce)], nonce)
copy(result[len(nonce):], ciphertext)
return result, nil
}


// decryptWithChaCha20Poly1305는 ChaCha20-Poly1305를 사용하여 데이터를 복호화합니다
func decryptWithChaCha20Poly1305(ciphertext []byte, key []byte) ([]byte, error)
if len(key) != 32 {
return nil, errors.New("키 길이는 32바이트여야 함")
}
// AEAD 암호 생성
aead, err := chacha20poly1305.New(key)
if err != nil {
return nil, err
}
```

```go
nonceSize := aead.NonceSize()
if len(ciphertext) < nonceSize {
return nil, errors.New("암호문이 너무 짧음")
}
// 논스와 암호문 분리
nonce := ciphertext[:nonceSize]
ciphertext = ciphertext[nonceSize:]
// 복호화 및 인증
plaintext, err := aead.Open(nil, nonce, ciphertext, nil)
if err != nil {
return nil, err
}
return plaintext, nil
}
```

2. 키 파생 함수

HKDF를 사용한 키 파생

```go
// DeriveKey는 HKDF를 사용하여 키를 파생합니다
func DeriveKey(masterKey []byte, salt []byte, info []byte, length int) ([]byte
if length <= 0 || length > 255*32 {
return nil, errors.New("잘못된 키 길이")
}
```

```go
// HKDF-Extract
h := hmac.New(sha256.New, salt)
h.Write(masterKey)
prk := h.Sum(nil)
// HKDF-Expand
var okm []byte
var counter byte = 1
for len(okm) < length {
h := hmac.New(sha256.New, prk)
h.Write(okm[len(okm)-32:]) // 이전 블록 (첫 번째는 빈 문자열)
h.Write(info)
h.Write([]byte{counter})
block := h.Sum(nil)
okm = append(okm, block...)
counter++
}
return okm[:length], nil
}
```

TLS 통신 보안

1. TLS 설정

노드 간 TLS 통신

```go
// TLSInfo는 노드의 TLS 정보를 포함합니다
type TLSInfo struct {
PubKey signature.PublicKey `json:"pub_key"`
}


// TLSAddress는 TLS 주소를 나타냅니다
type TLSAddress struct {
PubKey signature.PublicKey
IP net.IP
Port  uint16
}


// CreateTLSConfig는 노드 간 통신을 위한 TLS 설정을 생성합니다
func  CreateTLSConfig(identity *identity.Identity, peerPubKeys  map[signature.Pu
return &tls.Config{
// 클라이언트 인증서
Certificates: []tls.Certificate{*identity.TLSSigner.Certificate()},
// 서버 인증서 검증
VerifyPeerCertificate: func(rawCerts [][]byte, verifiedChains [][]*x509.Certific
if  len(rawCerts) == 0 {
return  errors.New("피어 인증서 없음")
}
```

```go
cert, err := x509.ParseCertificate(rawCerts[0])
if err != nil {
return err
}
// Ed25519 공개키 추출
pubKey, ok := cert.PublicKey.(ed25519.PublicKey)
if !ok {
return errors.New("잘못된 공개키 타입")
}
var sigPubKey signature.PublicKey
if err := sigPubKey.UnmarshalBinary(pubKey[:]); err != nil {
return err
}
// 허용된 피어인지 확인
if !peerPubKeys[sigPubKey] {
return errors.New("허용되지 않은 피어")
}
return nil
},
// TLS 1.3 강제
MinVersion: tls.VersionTLS13,
MaxVersion: tls.VersionTLS13,
// 클라이언트 인증 필수
```

```
ClientAuth: tls.RequireAndVerifyClientCert,
}, nil
}
```

2. 센트리 노드 TLS

센트리 클라이언트 설정

```
// NewSentryClient는 센트리 노드에 대한 TLS 클라이언트를 생성합니다
func NewSentryClient(sentryAddress TLSAddress, identity *identity.Identity) (*C
tlsConfig := &tls.Config{
// 센트리 노드 공개키만 신뢰
VerifyPeerCertificate: func(rawCerts [][]byte, verifiedChains [][]*x509.Certific
return verifySentryCertificate(rawCerts, sentryAddress.PubKey)
},
// 클라이언트 인증서 (센트리 전용)
Certificates: []tls.Certificate{*identity.TLSSentryClientCertificate},
// 서버 이름 검증 비활성화 (공개키 기반 검증 사용)
InsecureSkipVerify: true,
}
conn, err := tls.Dial("tcp", sentryAddress.String(), tlsConfig)
if err != nil {
return nil, err
}
```

```go
return &Client{conn: conn}, nil
}


func verifySentryCertificate(rawCerts [][]byte, expectedPubKey signature.Public
if len(rawCerts) == 0 {
return errors.New("센트리 인증서 없음")
}
cert, err := x509.ParseCertificate(rawCerts[0])
if err != nil {
return err
}
pubKey, ok := cert.PublicKey.(ed25519.PublicKey)
if !ok {
return errors.New("잘못된 센트리 공개키 타입")
}
var sigPubKey signature.PublicKey
if err := sigPubKey.UnmarshalBinary(pubKey[:]); err != nil {
return err
}
if !sigPubKey.Equal(expectedPubKey) {
return errors.New("센트리 공개키 불일치")
}
return nil
```

```
}
```

암호화 API 사용 예제

1. 완전한 서명 예제

```go
package main

import (
"crypto/rand"
"fmt"
"github.com/oasisprotocol/oasis-core/go/common/crypto/signature"
)

func main() {
// 1. 서명 컨텍스트 생성
ctx := signature.NewContext("oasis-core/example: test signing")
// 2. 키 쌍 생성
signer, err := signature.NewSigner(rand.Reader)
if err != nil {
panic(err)
}
defer signer.Reset()
```

```go
publicKey := signer.Public()

// 3. 메시지 서명

message := []byte("Hello, Oasis Core!")

sig, err := signer.ContextSign(ctx, message)

if  err != nil {

panic(err)

}

// 4. 서명 검증

err = signature.Verify(publicKey, ctx, message, sig)

if  err != nil {

panic(err)

}

fmt.Printf("서명 검증 성공!\n")

fmt.Printf("공개키: %s\n", publicKey.String())

fmt.Printf("서명: %x\n", sig)

}
```

2. 키 교환 예제

```go
package main


import (

"crypto/rand"
```

```go
"fmt"
"github.com/oasisprotocol/curve25519-voi/primitives/x25519"
)

func main() {
// 1. Alice 키 쌍 생성
alicePriv, alicePub, err := GenerateX25519KeyPair(rand.Reader)
if err != nil {
panic(err)
}
// 2. Bob 키 쌍 생성
bobPriv, bobPub, err := GenerateX25519KeyPair(rand.Reader)
if err != nil {
panic(err)
}
// 3. Alice가 공유 비밀 계산
aliceShared, err := ComputeSharedSecret(alicePriv, bobPub)
if err != nil {
panic(err)
}
// 4. Bob이 공유 비밀 계산
bobShared, err := ComputeSharedSecret(bobPriv, alicePub)
if err != nil {
```

```go
panic(err)
}
// 5. 공유 비밀 일치 확인
if *aliceShared == *bobShared {
fmt.Printf("키 교환 성공!\n")
fmt.Printf("공유 비밀: %x\n", aliceShared[:])
} else {
fmt.Printf("키 교환 실패!\n")
}
}
```

3. 해시 체인 예제

```go
package main

import (
"fmt"
"github.com/oasisprotocol/oasis-core/go/common/crypto/hash"
)

func  main() {
// 1. 초기 값
data := []byte("Genesis Block")
```

```go
// 2. 해시 체인 생성
var hashes []hash.Hash
for i := 0; i < 5; i++ {
h := hash.NewFromBytes(data)
hashes = append(hashes, h)
fmt.Printf("블록 %d 해시: %s\n", i, h.String())
// 다음 블록을 위해 이전 해시를 데이터로 사용
data = h[:]
}
// 3. 해시 체인 검증
fmt.Printf("\n해시 체인 검증:\n")
for i := 1; i < len(hashes); i++ {
expectedHash := hash.NewFromBytes(hashes[i-1][:])
if expectedHash.Equal(&hashes[i]) {
fmt.Printf("블록 %d -> %d: 유효\n", i-1, i)
} else {
fmt.Printf("블록 %d -> %d: 무효\n", i-1, i)
}
}
}
```

보안 고려사항

1. 키 관리 보안

- **안전한 난수 생성**: `crypto/rand` 패키지 사용 필수
- **키 제로화**: 사용 후 메모리에서 키 데이터 안전하게 삭제
- **키 저장**: 개인키의 안전한 저장과 접근 제어

2. 서명 보안

- **컨텍스트 분리**: 서로 다른 용도의 서명에 대한 도메인 분리
- **재생 공격 방지**: 논스와 타임스탬프 사용
- **배치 검증**: 성능 향상을 위한 배치 서명 검증

3. 통신 보안

- **TLS 1.3**: 최신 TLS 버전 사용 강제
- **상호 인증**: 클라이언트와 서버 모두 인증서 검증
- **공개키 고정**: 예상된 공개키만 신뢰

4. 암호화 보안

- **AEAD 사용**: 인증된 암호화로 무결성과 기밀성 동시 보장
- **키 파생**: 마스터 키에서 용도별 키 안전하게 파생
- **논스 관리**: 암호화 시 고유한 논스 사용

이러한 암호화 프로토콜을 통해 Oasis Core는 네트워크 전체의 보안을 보장하며, 각종 공격으로부터 시스템을 보호할 수 있습니다.